WEEK-1 TASK REPORT

Mathematical Customer Lifetime Value (CLV) Projection via BG/NBD Model from Scratch

Student Name: Harineeshwaran.V

Enrollment Number: TU6253210211020

Task Module: Mathematical Likelihood Estimation & Vectorized Optimization

Core Framework: BG/NBD Model (Beta-Geometric / Negative Binomial Distribution)

1. Task Design & Architectural Overview

Traditional CLV estimates rely on heuristic averages that break down for non-contractual, repeat-purchase businesses where customers can silently churn at any point. This module implements the BG/NBD (Beta-Geometric/Negative Binomial Distribution) model from first principles, deriving customer-level purchase and dropout behavior directly from the joint likelihood function rather than relying on pre-built marketing analytics packages such as lifetimes or BTYD.

2. Mathematical & Algorithmic Implementation

The BG/NBD model assumes each customer's transaction rate follows a Gamma(r, alpha) distribution and their probability of dropping out after a purchase follows a Beta(a, b) distribution. The log-likelihood for a customer with x repeat transactions, recency t_x, and observation period T is constructed and summed across the full transaction log, then maximized using SciPy's L-BFGS-B bounded quasi-Newton optimizer to recover the four heterogeneity parameters (r, alpha, a, b).

- **Assigned Module Focus:** Vectorized log-likelihood derivation and gradient-based parameter recovery
- **Execution Protocol:** Batched NumPy vectorization across millions of transaction rows to avoid per-customer Python loops

3. Python Simulation Script

Core code module engineered for this specific task specification:

```
import numpy as np
from scipy.special import gammaln
from scipy.optimize import minimize

class BGNBDModel:
def __init__(self):
self.r = self.alpha = self.a = self.b = None

def _log_likelihood(self, params, x, t_x, T):
r, alpha, a, b = params
term1 = gammaln(r + x) - gammaln(r) + r * np.log(alpha)
term2 = -(r + x) * np.log(alpha + T)
```

```
term3 = gammaln(a + b) + gammaln(b + x) - gammaln(b)
term3 -= gammaln(a + b + x)

term4 = np.where(
x &gt; 0,
np.log(a) - np.log(b + x - 1) - (r + x) * np.log(alpha + t_x),
-np.inf
)
max_term = np.maximum(term2, term4)
combined = max_term + np.log(
np.exp(term2 - max_term) + np.where(x &gt; 0, np.exp(term4 - max_term), 0)
)
return -np.sum(term1 + term3 + combined)

def fit(self, x, t_x, T, x0=(1.0, 1.0, 1.0, 1.0)):
bounds = [(1e-4, None)] * 4
result = minimize(
self._log_likelihood, x0, args=(x, t_x, T),
method="L-BFGS-B", bounds=bounds
)
self.r, self.alpha, self.a, self.b = result.x
return result
```

4. Simulation Results & Observations

- **Parameter Recovery:** Optimizer converged to stable (r, alpha, a, b) estimates within tolerance across repeated synthetic-data runs
- **Scalability:** Vectorized likelihood evaluation processed multi-million-row transaction logs without per-row Python iteration overhead

5. Conclusion

This component successfully fulfills the Week-1 task requirements, validating a from-scratch, vectorized implementation of the BG/NBD model's likelihood function and confirming that L-BFGS-B optimization reliably recovers transaction-rate and dropout-probability heterogeneity parameters for customer lifetime value projection.